

SRM ALPHA 300 · CATEGORY A

October DSA — Topics + Problem Variations

Cohort of 19 · Oct 1 Thu — Oct 31 Sat · 5 problems/day velocity

Build everything from scratch · no libraries beyond primitives · no AI tools · no per-problem URLs

How to read this doc

Each day lists ONE primary problem (the daily non-negotiable) and 5-6 variants that touch the same pattern from different angles. Variants are shown by name + angle only — students search for them on LeetCode or GFG themselves.

Each day also has a DON'T REACH FOR line — naming the specific library shortcut students will be tempted to use for that problem. Build it from primitives instead. That IS the lesson.

Each week opens with LeetCode tag URLs + GFG Explore filter URLs. Those are the only URLs in this doc.

The no-libraries rule

You must **BUILD** every data structure yourself. Importing a shortcut defeats the entire exercise.

ALLOWED — primitives only:

- list (your array — use `.append()`, `.pop()`, `[i]`, `len()`)
- dict (your hashmap — build it yourself with `[key]` access)
- set (your hashset — add, discard, membership)
- int, float, str, tuple, bool — primitives
- Basic str methods (`.isalnum()`, `.lower()`, `.split()`) are OK — they're str primitives, not library imports

FORBIDDEN — importing these = Cold List:

- `collections.deque` — build your queue from list + two pointers OR two stacks
- `collections.Counter` — build your frequency dict manually
- `collections.OrderedDict` — plain dict preserves order in Python 3.7+
- `collections.defaultdict` — use `dict.get(k, 0)` or if-check
- `heapq` — in December you'll BUILD a heap; until then don't use one
- `bisect` — write your own binary search in November
- `itertools.accumulate` / `takewhile` / `groupby` — one-line loops
- `functools.lru_cache` / `reduce` — write the memoization dict yourself
- `statistics.mean` / `mode` / `median` — one-line math
- `sorted()` and `list.sort()` in October — forbidden until November's sorting week (then you'll use your OWN sort functions)

Why:

The library IS the shortcut. Importing Counter when the problem asks you to build a frequency map is exactly like asking Claude to write the function. Both skip the thinking. Both get caught in viva.

In Sprints you'll use real libraries (Django, Redis clients, RabbitMQ clients). That's fine — they're infrastructure. DSA primitives are different: these are the structures those libraries are built FROM. You can't claim to understand Redis if you can't build a hashmap.

Problem budget for October

Primary problems in this plan: 30 (one per day, Oct 24 is test day with no primary)

Variants shown per day: 5 minimum, 6 maximum (most days show 5)

Variants required per day: 4 (student picks 4 from the list OR hunts own 4 from GFG/LeetCode category)

Total variants shown in this doc: 152

Monthly solve count — three brackets:

- MINIMUM (slacker tier · will land on Cold List): 1 primary + 2 variants per day = 90 problems
 - TARGET (locked floor · 5/day velocity): 1 primary + 4 variants per day = 150 problems
 - MAXIMUM (ambitious · Hot List territory): 1 primary + 6 variants per day = 210 problems
-

Daily rhythm · locked rules

- 1 primary problem per day — solved unaided, in Python, pushed to personal GitHub repo
 - 4 variants per day — hunted from this doc's list OR from the week's LeetCode/GFG category URL
 - Build every data structure from primitives — NO libraries beyond list/dict/set (see no-libraries rule above)
 - Push ALL 5 to GitHub: SA300-A{nn}-dsa / Oct / Week{N} / Day{DD}/
 - README per primary: approach (2-3 lines) · time + space complexity · one edge case missed on first attempt
 - README per variant: 1-line approach note
 - Discord: /done day{N} <5 github links> by midnight IST
 - ≥ 1 peer 👍 on any of the 5 by EOD
 - Streak counts only when all 5 are logged by midnight
 - Cold List trigger: 2 under-5 days in rolling 7-day window OR any library import OR any AI use → public flag in Discord
-

Oct 24 Sat · Pen & paper test day

3 problems picked by Iyappan from Days 1-23 pool. No laptops. Format: 90 min write on paper · 90 min peer-trace with new test inputs · 60 min public walkthrough.

Bonus drill: 'if your solution used list as a stack, be ready to defend what breaks if I take list away.' If a solution used Counter secretly, caught here.

WEEK 1 · Oct 1 Thu — Oct 7 Wed

Topic: Arrays + Hashmaps / Sets

Focus: Linear scan · index tracking · hash lookup ($O(n)$ time + $O(n)$ memory)

Where to find these problems — CATEGORY-LEVEL URLS ONLY

- LeetCode · Array tag — <https://leetcode.com/tag/array/>
- LeetCode · Hash Table tag — <https://leetcode.com/tag/hash-table/>
- GFG Explore · Arrays filter — <https://www.geeksforgeeks.org/explore?category=Arrays>
- GFG Explore · Hashing filter — <https://www.geeksforgeeks.org/explore?category=Hash>

Day 1 · Oct 1 Thu — Two Sum

Topic: Hashmap lookup

The canonical hashmap-for-complement pattern. Every candidate must know this cold.

DON'T REACH FOR: `collections.Counter`. Build the dict yourself with one pass.

PRIMARY (required)

• Two Sum

VARIANTS (6 shown · pick 4)

- [Sorted array + two-pointer] Two Sum II - Input Array Is Sorted
- [Three-sum variant (sorted + fixed i)] 3Sum
- [Target-sum closest] 3Sum Closest
- [Four pointers extension] 4Sum
- [Count pairs (less than)] Two Sum Less Than K
- [GFG pair with target] Pair with given sum in sorted array

Day 2 · Oct 2 Fri — Contains Duplicate

Topic: Set membership

Hashset entry point. Teaches the fundamental 'have I seen this?' pattern.

DON'T REACH FOR: `len(set(nums)) != len(nums)` one-liner. Build the set yourself and short-circuit on first dup.

PRIMARY (required)

• Contains Duplicate

VARIANTS (5 shown · pick 4)

- [Window-based duplicate] Contains Duplicate II
- [Bucket + window] Contains Duplicate III
- [Find all duplicates in $O(n)$ $O(1)$] Find All Duplicates in an Array
- [First repeating element] First Repeating Element
- [Array-position marker trick] Duplicate Zeros

Day 3 · Oct 3 Sat — Single Number

Topic: XOR / bit trick

Introduces XOR as a hash substitute. Opens the door to bit manipulation (December topic).

DON'T REACH FOR: `Counter`. The whole lesson is XOR — if you use a hashmap, you skipped it.

PRIMARY (required)

• Single Number

VARIANTS (5 shown · pick 4)

- [Three-times variant (bit count)] Single Number II
- [Two unknowns (XOR + partition)] Single Number III
- [Missing via XOR / sum] Missing Number
- [Char-difference via XOR / count] Find the Difference
- [Unique in sorted $O(\log n)$] Single Element in a Sorted Array

Day 4 · Oct 4 Sun — Running Sum of 1d Array

Topic: Prefix sum intro

Gentlest prefix-sum problem. Bridges to Week 3's prefix cluster.

DON'T REACH FOR: `itertools.accumulate()`. Write the one-line loop yourself.

PRIMARY (required)

• Running Sum of 1d Array

VARIANTS (5 shown · pick 4)

- [Static range queries] Range Sum Query - Immutable
- [Array transformation] Build Array from Permutation
- [Permutation reshuffle] Shuffle the Array
- [Compare per index + accumulate] Richest Customer Wealth
- [Left/right separate accumulations] Left and Right Sum Differences

Day 5 · Oct 5 Mon — Majority Element

Topic: Hashmap counting + Boyer-Moore

Two solutions: hashmap $O(n)$ $O(n)$, Boyer-Moore $O(n)$ $O(1)$. Teaches that algorithm selection matters.

DON'T REACH FOR: `Counter.most_common(1)`, `statistics.mode()`. Build both solutions by hand — the HashMap and Boyer-Moore versions.

PRIMARY (required)

• Majority Element

VARIANTS (5 shown · pick 4)

- [Multiple majorities ($>n/3$)] Majority Element II
- [Sorted array majority $O(\log n)$] Element Appearing More Than 25% in Sorted Array
- [Vote-count variant] Check If a Number Is Majority Element in a Sorted Array
- [Freq counting warm-up] Degree of an Array
- [Count-based classic] Majority Element (GFG)

Day 6 · Oct 6 Tue — Find All Numbers Disappeared in an Array

Topic: In-place marking ($O(1)$ space trick)

The 'use the array itself as a hashmap' trick. Interview-favorite space optimization.

DON'T REACH FOR: `set(range(1, n+1)) - set(nums)`. The lesson is the in-place marking trick for $O(1)$ space.

PRIMARY (required)

• Find All Numbers Disappeared in an Array

VARIANTS (5 shown · pick 4)

- [Single missing in $O(1)$] Missing Number
- [Find all duplicates] Find All Duplicates in an Array
- [First positive missing] First Missing Positive
- [Mismatched pair] Set Mismatch
- [GFG missing number] Missing number in array

Day 7 · Oct 7 Wed — Intersection of Two Arrays II

Topic: Hashmap counting + multiset

Multiset intersection — teaches when a Counter beats a Set.

DON'T REACH FOR: `collections.Counter` and `&-intersection`. Build two dicts manually, decrement during walk.

PRIMARY (required)

• Intersection of Two Arrays II

VARIANTS (5 shown · pick 4)

- [Set intersection (unique)] Intersection of Two Arrays
- [Union with duplicates] Union of Two Arrays
- [Char-level intersection across k lists] Find Common Characters
- [Sorted-array two-pointer intersect] Intersection of 3 Sorted Arrays

- [Diff of two arrays] Find the Difference of Two Arrays

WEEK 2 · Oct 8 Thu — Oct 14 Wed

Topic: Two Pointers + Sliding Window

Focus: Pointers from both ends OR same direction · fixed window · variable window

Where to find these problems — CATEGORY-LEVEL URLS ONLY

- LeetCode · Two Pointers tag — <https://leetcode.com/tag/two-pointers/>
- LeetCode · Sliding Window tag — <https://leetcode.com/tag/sliding-window/>
- GFG Explore · Two Pointers filter — <https://www.geeksforgeeks.org/explore?category=Two%20Pointers>
- GFG Explore · Sliding Window filter — <https://www.geeksforgeeks.org/explore?category=Sliding%20Window>

Day 8 · Oct 8 Thu — Valid Palindrome

Topic: Two pointers from ends

Simplest two-pointer problem. String cleanup + opposite-end pointers.

DON'T REACH FOR: `s == s[::-1]`. Also skip the `".join(c for c in s if c.isalnum()).lower()` one-liner — do the cleanup inline during the two-pointer walk.

PRIMARY (required)

• Valid Palindrome

VARIANTS (5 shown · pick 4)

- [Allow 1 deletion (greedy skip)] Valid Palindrome II
- [Palindrome on linked list] Palindrome Linked List
- [Count palindromic substrings] Palindromic Substrings
- [Skip non-alpha] Reverse Only Letters
- [Number palindrome without string] Palindrome Number

Day 9 · Oct 9 Fri — Reverse String

Topic: In-place swap

Simplest in-place two-pointer swap. Also recursion warm-up for November.

DON'T REACH FOR: `s.reverse()`, `reversed(s)`, `s[::-1]`. Write the two-pointer swap loop. The lesson IS the loop.

PRIMARY (required)

• Reverse String

VARIANTS (5 shown · pick 4)

- [k-chunk reversal] Reverse String II
- [Reverse only vowels] Reverse Vowels of a String
- [Reverse words preserving order] Reverse Words in a String III
- [Prefix-only reversal] Reverse Prefix of Word
- [Full sentence reversal] Reverse Words in a String

Day 10 · Oct 10 Sat — Remove Duplicates from Sorted Array

Topic: Two pointers same direction (read + write)

The read-pointer + write-pointer idiom. Shows up in dozens of in-place problems.

DON'T REACH FOR: `list(set(nums))` or `list(dict.fromkeys(nums))`. The lesson is the in-place read-write pointer idiom.

PRIMARY (required)

• Remove Duplicates from Sorted Array

VARIANTS (5 shown · pick 4)

- [Allow at most 2] Remove Duplicates from Sorted Array II
- [Remove specific value] Remove Element
- [Zeroes to end] Move Zeroes
- [Partition by parity] Sort Array By Parity
- [Dutch national flag (3-way)] Sort Colors

Day 11 · Oct 11 Sun — Merge Sorted Array

Topic: Two pointers from the end (in-place merge)

The 'merge from the back to avoid overwrites' trick. Also sets up merge sort in November.

DON'T REACH FOR: `sorted(nums1 + nums2)`, `heapq.merge()`. The lesson is the three-pointer merge from the END, in-place.

PRIMARY (required)

• Merge Sorted Array

VARIANTS (5 shown · pick 4)

- [Merge sorted linked lists] Merge Two Sorted Lists
- [Merge k lists (heap bridge)] Merge k Sorted Lists
- [Overlapping intervals merge] Merge Intervals
- [Interval insertion] Insert Interval
- [Convert to RLE then merge] Decompress Run-Length Encoded List

Day 12 · Oct 12 Mon — Squares of a Sorted Array

Topic: Two pointers from both ends (compare absolute values)

Sneaky two-pointer: biggest square is at one end, not the middle. Teaches 'where is the extremum?'

DON'T REACH FOR: `sorted(x*x for x in nums)`. The point is to achieve $O(n)$ by exploiting the sorted input — not $O(n \log n)$.

PRIMARY (required)

• Squares of a Sorted Array

VARIANTS (5 shown · pick 4)

- [Sort by parity (two-ptr swap)] Sort Array By Parity II
- [Minimum sort window] Shortest Unsorted Continuous Subarray
- [Compare pairs after sort] Height Checker
- [Freq-based ordering] Relative Sort Array
- [Max sum of absolute diffs] Maximum Absolute Sum of Any Subarray

Day 13 · Oct 13 Tue — Maximum Average Subarray I

Topic: Fixed-size sliding window

The window-of-size-k template. Slide one in, one out. Keep running sum.

DON'T REACH FOR: `sum(nums[i:i+k])` inside the loop (that makes it $O(n*k)$). Keep a running sum, add one, subtract one.

PRIMARY (required)

• Maximum Average Subarray I

VARIANTS (6 shown · pick 4)

- [Max sum of k elements] Max Sum Subarray of Size K
- [Variable window (shrink)] Minimum Size Subarray Sum
- [Window of distinct chars] Longest Substring Without Repeating Characters
- [Freq-match window] Find All Anagrams in a String
- [Boolean-check window] Kids With the Greatest Number of Candies
- [Max consecutive after flips] Max Consecutive Ones III

Day 14 · Oct 14 Wed — Contains Duplicate II

Topic: Sliding window + hashmap

Combines Week 1 hashmaps with Week 2 windows. Keep a window of 'what have I seen in the last k?'

DON'T REACH FOR: Counter + max frequency check. Keep a rolling dict of last-seen-index per value, update it inline.

PRIMARY (required)

• Contains Duplicate II

VARIANTS (5 shown · pick 4)

- [Bucket-sort window] Contains Duplicate III
- [Permutation detection] Permutation in String
- [Window + char swap check] Find All Anagrams in a String

- [Freq-based window] Longest Repeating Character Replacement
- [Sliding stats] Moving Average from Data Stream

WEEK 3 · Oct 15 Thu — Oct 21 Wed

Topic: Strings + Prefix Sum

Focus: Character counting · index arithmetic · cumulative sums

Where to find these problems — CATEGORY-LEVEL URLS ONLY

- LeetCode · String tag — <https://leetcode.com/tag/string/>
- LeetCode · Prefix Sum tag — <https://leetcode.com/tag/prefix-sum/>
- GFG Explore · Strings filter — <https://www.geeksforgeeks.org/explore?category=Strings>
- GFG Explore · Prefix Sum filter — <https://www.geeksforgeeks.org/explore?category=prefix-sum>

Day 15 · Oct 15 Thu — Valid Anagram

Topic: Character frequency counting

The Counter/dict pattern for strings. Opens the door to grouping, matching, and window problems.

DON'T REACH FOR: Counter(s) == Counter(t). Also skip sorted(s) == sorted(t). Build two dicts by hand and compare keys + values.

PRIMARY (required)

• Valid Anagram

VARIANTS (5 shown · pick 4)

- [Group by anagram signature] Group Anagrams
- [Window + anagram] Find All Anagrams in a String
- [Char permutation detection] Permutation in String
- [Target-cover window] Minimum Window Substring
- [Char-count equality variant] Determine if Two Strings Are Close

Day 16 · Oct 16 Fri — First Unique Character in a String

Topic: Freq map + ordered scan

Two-pass pattern: count, then scan. Trains the 'build a map first, then query it' habit.

DON'T REACH FOR: Counter(s), OrderedDict. Use a plain dict + two-pass loop. First pass counts, second pass finds the first with count == 1.

PRIMARY (required)

• First Unique Character in a String

VARIANTS (5 shown · pick 4)

- [Two bags, find odd one out] Find the Difference
- [First repeat in a stream] First Letter to Appear Twice
- [Freq-array equality] Unique Number of Occurrences
- [Sort by frequency] Sort Characters By Frequency
- [Freq + stability sort] Increasing Decreasing String

Day 17 · Oct 17 Sat — Longest Common Prefix

Topic: Vertical scan / horizontal scan

Column-wise scan — a surprising pattern. Also sets up tries (December).

DON'T REACH FOR: os.path.commonprefix(), itertools.takewhile(), zip(*strs) + set tricks. Walk the characters by index, bail on first mismatch.

PRIMARY (required)

• Longest Common Prefix

VARIANTS (5 shown · pick 4)

- [Char overlap across lists] Find Common Characters
- [Prefix in two words] Check If a Word Occurs As Prefix of Any Word
- [Compare after rotation] Rotate String
- [Char-by-char swap check] Buddy Strings
- [Substring match] Repeated Substring Pattern

Day 18 · Oct 18 Sun — Reverse Words in a String III**Topic:** String split + in-place reverse

Combines Week 2 reversal with string tokenization. Preps the common 'reverse everything, then reverse parts' trick.

DON'T REACH FOR: `' '.join(w[::-1] for w in s.split())`. Build a list, walk char-by-char, reverse each word in-place using your Day 9 swap.**PRIMARY (required)**• **Reverse Words in a String III****VARIANTS (5 shown · pick 4)**

- [Reverse whole sentence] [Reverse Words in a String](#)
- [Reverse ignoring non-alpha] [Reverse Only Letters](#)
- [Last word length] [Length of Last Word](#)
- [Casing transformation] [To Lower Case](#)
- [Compression] [String Compression](#)

Day 19 · Oct 19 Mon — Isomorphic Strings**Topic:** Bijective hashmap (two dicts)

The 'map both directions' insight. Catches a subtle bug: one-way map isn't enough.

DON'T REACH FOR: `len(set(zip(s,t))) == len(set(s)) == len(set(t))`. Build two dicts explicitly and defend why both are needed.**PRIMARY (required)**• **Isomorphic Strings****VARIANTS (5 shown · pick 4)**

- [Pattern → words bijection] [Word Pattern](#)
- [Multiple words share pattern] [Find and Replace Pattern](#)
- [Pair-swap single] [Buddy Strings](#)
- [Cyclic rotation equivalence] [Rotate String](#)
- [Char equivalence groups] [Groups of Special-Equivalent Strings](#)

Day 20 · Oct 20 Tue — Find Pivot Index**Topic:** Prefix sum — split point

Prefix sum made concrete: can you split the array where left sum == right sum?

DON'T REACH FOR: `itertools.accumulate()` for the prefix. `sum(nums[:i])` inside the loop ($O(n^2)$). Keep total + running left in $O(n)$.**PRIMARY (required)**• **Find Pivot Index****VARIANTS (5 shown · pick 4)**

- [Middle index variant] [Find the Middle Index in Array](#)
- [GFG equilibrium point] [Equilibrium Point](#)
- [Minimum step so prefix never goes negative] [Minimum Value to Get Positive Step by Step Sum](#)
- [Left/right separate sums] [Left and Right Sum Differences](#)
- [Running sum (prerequisite)] [Running Sum of 1d Array](#)

Day 21 · Oct 21 Wed — Range Sum Query - Immutable**Topic:** Prefix sum — static range queries

Build-once, query-many pattern. Teaches when precomputation pays off.

DON'T REACH FOR: `itertools.accumulate()` for the prefix array. Build it with a loop in `__init__`.**PRIMARY (required)**• **Range Sum Query - Immutable****VARIANTS (5 shown · pick 4)**

- [2D prefix sum] [Range Sum Query 2D - Immutable](#)
- [Prefix + hashmap] [Subarray Sum Equals K](#)
- [Prefix mod sum] [Continuous Subarray Sum](#)
- [Prefix + suffix combined] [Product of Array Except Self](#)

- [Mutable version (segment tree bridge)] **Range Sum Query - Mutable**

WEEK 4 · Oct 22 Thu — Oct 28 Wed

Topic: Stacks + Queues

Focus: LIFO / FIFO · bracket matching · monotonic stack intro

Where to find these problems — CATEGORY-LEVEL URLS ONLY

- LeetCode · Stack tag — <https://leetcode.com/tag/stack/>
- LeetCode · Queue tag — <https://leetcode.com/tag/queue/>
- LeetCode · Monotonic Stack tag — <https://leetcode.com/tag/monotonic-stack/>
- GFG Explore · Stack filter — <https://www.geeksforgeeks.org/explore?category=Stack>
- GFG Explore · Queue filter — <https://www.geeksforgeeks.org/explore?category=Queue>

Day 22 · Oct 22 Thu — Valid Parentheses

Topic: Stack-based matching

The canonical 'push open, pop and compare on close' pattern. One of the top-5 interview questions ever.

DON'T REACH FOR: collections.deque. A plain list is your stack — list.append() to push, list.pop() to pop. The whole lesson is 'what IS a stack.'

PRIMARY (required)

• Valid Parentheses

VARIANTS (5 shown · pick 4)

- [Insert to make valid] Minimum Add to Make Parentheses Valid
- [Strip outer layer] Remove Outermost Parentheses
- [Scored nesting] Score of Parentheses
- [Length of longest valid] Longest Valid Parentheses
- [Deletion variant] Minimum Remove to Make Valid Parentheses

Day 23 · Oct 23 Fri — Baseball Game

Topic: Stack as running state

Simplest 'simulate a sequence using a stack' problem. Opens the door to real stack reasoning.

DON'T REACH FOR: deque. Use list. Also don't use sum(stack) inside the loop — keep a running total as you push/pop.

PRIMARY (required)

• Baseball Game

VARIANTS (5 shown · pick 4)

- [Build via stack ops] Build an Array With Stack Operations
- [Collapse adjacent] Make The String Great
- [Monotonic stack intro] Final Prices With Special Discount
- [Adjacent duplicate removal] Remove All Adjacent Duplicates In String
- [Delete + apply backspace] Backspace String Compare

Day 24 · Oct 24 Sat · ⚡ PEN & PAPER TEST DAY — (no new problem — 3 problems picked from Days 1-23)

Topic: Test day

90 min write on paper · 90 min peer-trace with new inputs · 60 min public walkthrough.

Day 25 · Oct 25 Sun — Min Stack

Topic: Augmented stack (keep a 2nd stack for min)

Teaches 'carry extra state alongside the primary stack'. Pattern reappears in max-stack, LRU, etc.

DON'T REACH FOR: heapq. If you use a heap, getMin becomes $O(\log n)$ and the WHOLE lesson is lost. Target: all 4 ops $O(1)$ using two lists.

PRIMARY (required)

• Min Stack

VARIANTS (5 shown · pick 4)

- [Max variant] Max Stack

- [Design with increment op] [Design a Stack With Increment Operation](#)
- [Monotonic stack use case] [Online Stock Span](#)
- [Replay stack sequence] [Validate Stack Sequences](#)
- [Smallest element queue] [Sliding Window Maximum](#)

Day 26 · Oct 26 Mon — Next Greater Element I

Topic: Monotonic decreasing stack

The gateway to monotonic stacks. Interview goldmine — appears in temperature, histogram, stock span.

DON'T REACH FOR: deque, bisect. Use a plain list as the monotonic stack. Push conditionally, pop while top < current.

PRIMARY (required)

• Next Greater Element I

VARIANTS (5 shown · pick 4)

- [Circular array] [Next Greater Element II](#)
- [Rearrange digits (number theory + stack)] [Next Greater Element III](#)
- [Wait days version] [Daily Temperatures](#)
- [Histogram rectangles] [Largest Rectangle in Histogram](#)
- [Rolling max via deque] [Sliding Window Maximum](#)

Day 27 · Oct 27 Tue — Implement Queue using Stacks

Topic: Two-stack FIFO simulation

Design problem: prove FIFO using only LIFO. Amortized $O(1)$ analysis.

DON'T REACH FOR: deque. Using deque trivializes the ENTIRE problem — it already IS a queue. The whole exercise is simulating a queue when all you have is two lists (stacks).

PRIMARY (required)

• Implement Queue using Stacks

VARIANTS (5 shown · pick 4)

- [Reverse direction] [Implement Stack using Queues](#)
- [Fixed-size ring] [Design Circular Queue](#)
- [Two-ended fixed ring] [Design Circular Deque](#)
- [Rolling average via queue] [Moving Average from Data Stream](#)
- [Tweet feed design] [Design Twitter](#)

Day 28 · Oct 28 Wed — Number of Recent Calls

Topic: Monotonic queue (sliding window of time)

Queue + time-sliding — exactly the shape that reappears in Sprint 3's rate limiter.

DON'T REACH FOR: deque, bisect. Use a plain list and pop(0) when old (yes, $O(n)$ — then optimize to a fixed-size circular buffer and compare).

PRIMARY (required)

• Number of Recent Calls

VARIANTS (5 shown · pick 4)

- [Time-window counter design] [Design Hit Counter](#)
- [Rate limit per message] [Logger Rate Limiter](#)
- [Rolling max window] [Sliding Window Maximum](#)
- [Rolling average] [Moving Average from Data Stream](#)
- [In/out event tracking] [Design Browser History](#)

WEEK 5 · Oct 29 Thu — Oct 31 Sat

Topic: Consolidation

Focus: Cross-pattern review before November (recursion + linked lists + binary search + sorting)

Where to find these problems — CATEGORY-LEVEL URLS ONLY

- LeetCode · revisit any weak tag — <https://leetcode.com/problemset/all/>
- GFG Explore · revisit any weak category — <https://www.geeksforgeeks.org/explore>

Day 29 · Oct 29 Thu — Best Time to Buy and Sell Stock

Topic: Single-pass tracking (arrays revisit)

Track running min + max profit in one pass. Simple but introduces greedy intuition for December.

DON'T REACH FOR: min() and max() calls inside the loop. Keep two variables: running_min, best_profit. One pass.

PRIMARY (required)

• Best Time to Buy and Sell Stock

VARIANTS (5 shown · pick 4)

- [Multiple transactions (greedy)] Best Time to Buy and Sell Stock II
- [At most 2 transactions] Best Time to Buy and Sell Stock III
- [Cooldown constraint] Best Time to Buy and Sell with Cooldown
- [Transaction fee] Best Time to Buy and Sell with Transaction Fee
- [Max subarray — related pattern] Maximum Subarray

Day 30 · Oct 30 Fri — Move Zeroes

Topic: Two pointers revisit (read + write)

Classic in-place partition. Same muscle as Remove Duplicates.

DON'T REACH FOR: nums.sort(key=lambda x: x==0), filter(), [x for x in nums if x!=0] + padding. The POINT is in-place with O(1) extra space.

PRIMARY (required)

• Move Zeroes

VARIANTS (5 shown · pick 4)

- [Remove a specific value] Remove Element
- [Duplicates in sorted] Remove Duplicates from Sorted Array
- [Partition by parity] Sort Array By Parity
- [Partition by predicate] Partition Array According to Given Pivot
- [In-place rotate] Rotate Array

Day 31 · Oct 31 Sat — Backspace String Compare

Topic: Stacks revisit (two stacks compared)

Build two stacks via a simulated backspace, then compare. Easy to code, satisfying to drill.

DON'T REACH FOR: deque. Use list as stack. Bonus: solve the O(1) space version using two reverse pointers.

PRIMARY (required)

• Backspace String Compare

VARIANTS (5 shown · pick 4)

- [Simplify a path] Simplify Path
- [Remove adjacent dupes] Remove All Adjacent Duplicates In String
- [Remove k digits greedily] Remove K Digits
- [Reverse ignoring non-alpha] Reverse Only Letters
- [Decode run-length] Decoded String at Index

Monthly totals

Primary problems: 30 (one per day; Oct 24 is test day)

Variants shown in this doc: 152 (across 30 days)

Variants range per day: 5 minimum · 6 maximum · 5 most days

Student solve expectations:

- MINIMUM (will trip Cold List): 1 primary + 2 variants = ~90 solves/month
- TARGET (locked floor at 5/day): 1 primary + 4 variants = 150 solves/month
- MAXIMUM (Hot List): 1 primary + 6 variants = 210 solves/month

Pen & paper test: Oct 24 Sat (3 problems from Days 1-23)